Title: Develop a Customer Order Management System to Retrieve Data Using SQL Joins

Aim: To learn and implement inner and outer joins in SQL for retrieving data from related tables in a customer order management system.

Prerequisite: Basic knowledge of SQL, including creating tables, inserting data, and understanding relational databases. Familiarity with SELECT statements and database schemas involving primary and foreign keys.

Hardware Requirement

- Computer with minimum 4GB RAM
- Internet connection

Software Requirement

- MySQL / PostgreSQL
- MySQL Workbench / pgAdmin
- Windows / Linux OS

Theory: SQL joins are used to combine rows from two or more tables based on a related column between them. This is essential in relational databases like a Customer Order Management System, where data is spread across tables such as Customers and Orders. ● INNER JOIN: Returns only the rows where there is a match in both tables. It retrieves data for customers who have placed orders. This join is the most common type used when you want to exclude unmatched records from the result set. It can be specified using the INNER JOIN keyword or simply JOIN in most SQL dialects. Performance-wise, INNER JOINs are often efficient as they filter out non-matching rows early in the query execution. An example scenario is generating a report of active customers based on their order history. ● LEFT JOIN (or LEFT OUTER JOIN): Returns all rows from the left table (e.g., Customers) and the matched rows from the right table (e.g., Orders). If no match, NULL values are returned for the right table columns. This includes customers who haven't placed orders. It preserves all records from the primary (left) table, making it ideal for identifying gaps in data relationships. In query optimization, LEFT JOINs may require more resources if the left table is large and unmatched rows are numerous. This join type is particularly useful in analytics for customer retention studies. For instance, it can list all customers and highlight those with zero orders for targeted marketing. ● RIGHT JOIN (or RIGHT OUTER JOIN): Returns all rows from the right table and the matched rows from the left table. If no match, NULL values are returned for the left table columns. This includes orders not linked to existing customers. It mirrors the LEFT JOIN but prioritizes the right table, which can be rewritten as a LEFT JOIN by swapping table order for better readability. RIGHT JOINs are less commonly used due to convention but essential in scenarios where the secondary table's completeness is key. They help in

auditing data integrity, such as finding orphan orders without associated customer records. An application might involve reconciling imported order data with an existing customer database. ● FULL OUTER JOIN: Returns all rows when there is a match in either table. If no match, NULL values are returned for the non-matching table's columns. This provides a complete list of customers and orders, including unmatched ones. It combines the effects of both LEFT and RIGHT JOINs, offering a full union of datasets. However, FULL OUTER JOINs can produce large result sets and may impact performance in big databases. They are valuable for data merging tasks, like synchronizing disparate systems in a management application. For example, it can reveal both inactive customers and unassigned orders in one query for comprehensive cleanup.

Steps

Step 1: Create Database

```
CREATE DATABASE CustomerDB;
USE CustomerDB;
```

Step 2: Create Tables

```
CREATE TABLE Customers(
 CustomerID INT PRIMARY KEY,
 CustomerName VARCHAR(50),
 Email VARCHAR(50),
 City VARCHAR(50)
);

CREATE TABLE Orders(
 OrderID INT PRIMARY KEY,
 OrderDate DATE,
 Amount DECIMAL(10,2),
 CustomerID INT
);
```

Step 3: Insert Sample Data

```
INSERT INTO Customers VALUES
(1,'Ali','ali@gmail.com','Pune'),
(2,'Sara','sara@gmail.com','Mumbai'),
(3,'John','john@gmail.com','Delhi');

INSERT INTO Orders VALUES
(101,'2025-01-10',5000,1),
(102,'2025-01-15',3000,2),
(103,'2025-01-20',2500,5);
```

Step 4: INNER JOIN

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
INNER JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Step 5: LEFT JOIN

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Step 6: RIGHT JOIN

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
RIGHT JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Step 7: FULL OUTER JOIN

(PostgreSQL)

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
FULL OUTER JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Conclusion: This experiment demonstrates how SQL joins enable efficient data retrieval from related tables in a customer order management system. INNER JOIN is useful for matched data, while outer joins (LEFT, RIGHT, FULL) handle scenarios with unmatched records, ensuring comprehensive querying without data loss.

Assignment

1.Explain Joins in SQL.

2.Illustrate Inner and Left Joins.

3.Explain which joins shows unmatched rows.

4.Explain Full Outer joins.

5.Explain why do we use foreign Keys for Joins.